

Glossário de Termos — Aula 04

Restringindo e Classificando Dados

Nesta aula você vai aprender a filtrar linhas e ordenar resultados em SQL. Os termos a seguir formam o vocabulário de toda consulta seletiva — vão acompanhar você em praticamente 100% das consultas reais que escreverá daqui para frente. Mantenha este glossário por perto durante os exercícios, especialmente nos primeiros dias.

1. Cláusula WHERE e Filtragem de Linhas

WHERE (Cláusula de filtragem)

Cláusula que restringe quais linhas serão exibidas no resultado da consulta. Aparece logo após FROM e contém uma condição — uma expressão lógica que cada linha precisa satisfazer para passar pelo filtro.

Analogia: É como o porteiro de uma festa com lista: ele checa cada pessoa que chega contra a regra ('está na lista?'), e só deixa entrar quem cumpre o critério.

```
SELECT last_name, salary
FROM   employees
WHERE  department_id = 90;
```

Dica: A ordem das cláusulas é fixa: SELECT, FROM, WHERE, ORDER BY. WHERE nunca vem antes de FROM nem depois de ORDER BY.

Condição (Predicate / Expressão lógica)

Expressão dentro do WHERE que produz um valor lógico (verdadeiro ou falso) para cada linha. Quando o resultado é verdadeiro, a linha entra no resultado; quando é falso (ou NULL), a linha é descartada.

Analogia: É a pergunta de sim/não feita a cada linha: 'esse funcionário ganha mais de 5000?'. Só passa quem responde sim.

```
WHERE salary > 5000
WHERE last_name = 'King'
WHERE hire_date > '01-JAN-95'
```

Strings de Caracteres e Datas (Aspas simples ' ')

Em SQL, valores de texto e datas precisam ficar entre aspas simples. Strings são sensíveis a maiúsculas e minúsculas: 'King' é diferente de 'KING'. O formato padrão de data no Oracle é DD-MON-RR (ex: '17-FEB-96').

Analogia: Aspas simples funcionam como o sinal de citação numa redação: marcam onde começa e termina o valor literal.

```
-- Texto:
WHERE last_name = 'Silva';
-- Data:
WHERE hire_date = '01-JAN-95';
```

Dica: Confundir aspas simples (valor) com aspas duplas (apelido) é um erro de iniciante comum. Em SQL: aspas simples para dados; aspas duplas para nomes de coluna.

2. Condições de Comparação

= (Igualdade) (Igual a)

Compara se dois valores são exatamente iguais. Funciona com números, strings e datas. Para strings, a comparação é caractere a caractere e considera maiúsculas e minúsculas.

Analogia: É como conferir se o número da senha digitada bate exatamente com o da ficha — qualquer dígito diferente já invalida.

```
WHERE department_id = 90;
WHERE last_name = 'King';
```

Dica: Para comparar com NULL nunca use =. Use IS NULL — '= NULL' jamais retorna verdadeiro.

> (Maior que) e >= (Maior ou igual) (Operadores de ordem (acima))

Testam se o valor da esquerda é maior (ou maior ou igual) ao da direita. Para datas, 'maior' significa mais recente; para strings, significa alfabeticamente depois.

Analogia: É a régua de altura no parque: só passa quem tem altura maior (ou igual) ao limite marcado.

```
WHERE salary > 5000;
WHERE hire_date >= '01-JAN-2000';
```

< (Menor que) e <= (Menor ou igual) (Operadores de ordem (abaixo))

Testam se o valor da esquerda é menor (ou menor ou igual) ao da direita. Para datas, 'menor' significa mais antigo; para strings, alfabeticamente antes.

Analogia: É o limite de velocidade na via: tudo abaixo (ou igual) ao limite está dentro da regra.

```
WHERE salary <= 3000;
WHERE last_name < 'M';
```

<> ou != (Diferente de) (Negação de igualdade)

Testam se dois valores são diferentes. As duas escritas (<> e !=) têm exatamente o mesmo significado — escolha uma e mantenha o padrão no código.

Analogia: É o filtro 'todos menos um' da lista de presenças: mostra todos os alunos exceto o aluno escolhido.

```
WHERE job_id <> 'IT_PROG';
WHERE department_id != 90;
```

Dica: A forma padrão ANSI é <>; o != funciona em quase todos os SGBDs mas é considerada uma extensão. Prefira <> em código portátil.

3. Outras Condições de Comparação

BETWEEN ... AND (Intervalo inclusivo)

Verifica se um valor está dentro de uma faixa, incluindo os limites inferior e superior. Substitui combinações de $\geq$ e $\leq$ com AND, deixando o código mais limpo. Funciona com números, datas e strings.

Analogia: É a faixa salarial num anúncio de emprego: 'salário entre R\$ 3.000 e R\$ 5.000' — os dois extremos contam.

```
WHERE salary BETWEEN 2500 AND 3500;  
-- equivale a:  
WHERE salary >= 2500 AND salary <= 3500;
```

Dica: Sempre informe o MENOR valor primeiro: BETWEEN 3500 AND 2500 não retorna nenhuma linha.

IN (Lista de valores possíveis)

Verifica se um valor está presente em uma lista de opções. Substitui várias condições OR encadeadas, deixando a consulta muito mais legível. Use NOT IN para excluir os valores da lista.

Analogia: É como a lista VIP da festa: 'pode entrar quem se chama João, Maria ou Pedro' — qualquer um dos três passa.

```
WHERE manager_id IN (100, 101, 201);  
WHERE job_id NOT IN ('IT_PROG', 'ST_CLERK');
```

Dica: Strings dentro do IN também ficam entre aspas simples. Números, não.

LIKE (Busca por padrão de texto)

Filtra strings com base em padrões, não em igualdade exata. Usa dois caracteres curinga: % (porcentagem) representa qualquer sequência de caracteres (zero ou mais), e _ (sublinhado) representa exatamente um caractere qualquer.

Analogia: É a busca por palavra-chave do navegador: 'tudo que começa com Mar' acha Mariana, Marcelo, Maria — sem precisar saber o nome inteiro.

```
-- Sobrenomes que comecam com 'S':  
WHERE last_name LIKE 'S%';  
  
-- Segunda letra e 'o':  
WHERE last_name LIKE '_o%';  
  
-- Termina com 'er':  
WHERE last_name LIKE '%er';
```

Dica: Para buscar o próprio caractere % ou _ literalmente, use ESCAPE: LIKE 'SA_%' ESCAPE '\'

4. Condições com NULL

IS NULL (Testa ausência de valor)

Verifica se uma coluna está vazia (NULL). Como NULL representa ausência de valor, não pode ser comparado com = ou <>. O operador IS NULL é a única forma correta de testar nulos em SQL padrão.

Analogia: É a pergunta 'esse campo do formulário ficou em branco?'. Branco não é zero nem espaço — é simplesmente não preenchido.

```
-- Funcionarios SEM comissao:
WHERE commission_pct IS NULL;
```

Dica: Errado: WHERE commission_pct = NULL → nunca retorna nenhuma linha. Correto: WHERE commission_pct IS NULL.

IS NOT NULL (Testa presença de valor)

Verifica se uma coluna tem algum valor preenchido (não-nulo). É o oposto de IS NULL. Útil para filtrar registros completos antes de fazer cálculos que seriam contaminados por NULL.

Analogia: É o filtro 'mostrar só fichas com telefone preenchido' do cadastro.

```
-- Funcionarios COM comissao:
WHERE commission_pct IS NOT NULL;
```

5. Operadores Lógicos

AND (E lógico)

Conecta duas ou mais condições. Para a linha passar pelo filtro, TODAS as condições conectadas por AND precisam ser verdadeiras. Se ao menos uma falha, a linha é descartada.

Analogia: É a checagem dupla no aeroporto: você precisa ter passagem E documento. Só uma das duas não basta.

```
WHERE salary >= 5000
      AND job_id = 'IT_PROG';
```

Dica: Em consultas com muitos AND, indente cada condição em uma linha — fica muito mais fácil de ler e depurar.

OR (OU lógico)

Conecta duas ou mais condições. Para a linha passar, BASTA QUE UMA das condições conectadas seja verdadeira. Funciona como o 'ou' inclusivo: se as duas forem verdadeiras, também aceita.

Analogia: É o cupom de desconto 'aniversariantes do mês OU clientes VIP': basta cumprir uma das duas para ganhar.

```
WHERE salary >= 10000
      OR job_id = 'IT_PROG';
```

NOT (Negação lógica)

Inverte o resultado de uma condição. Se a condição era verdadeira, vira falsa; se era falsa, vira verdadeira. Costuma ser usado junto com IN, BETWEEN, LIKE e NULL para criar 'condições negativas'.

Analogia: É o botão 'inverter seleção' do gerenciador de arquivos: o que estava marcado fica desmarcado e vice-versa.

```
WHERE job_id NOT IN ('IT_PROG', 'ST_CLERK');
WHERE salary NOT BETWEEN 2500 AND 3500;
WHERE commission_pct IS NOT NULL;
```

Precedência de Operadores Lógicos (Quem é avaliado primeiro?)

Quando AND e OR aparecem na mesma cláusula, AND é avaliado ANTES de OR. NOT vem antes dos dois. Para alterar essa ordem ou tornar a intenção clara, use parênteses — eles têm prioridade absoluta.

Analogia: É a regra de matemática 'multiplicação antes de soma'. AND funciona como multiplicação; OR funciona como soma. Parênteses sobrepõem a regra.

```
-- Cuidado: AND e OR juntos
WHERE job_id = 'SA_REP'
      OR job_id = 'AD_PRES'
      AND salary > 15000;
-- Equivale a:
WHERE job_id = 'SA_REP'
      OR (job_id = 'AD_PRES' AND salary > 15000);
```

Dica: Sempre que misturar AND e OR, use parênteses para deixar a intenção explícita — evita bugs sutis e ajuda quem ler o código depois.

6. Cláusula ORDER BY

ORDER BY (Classificação do resultado)

Define a ordem em que as linhas aparecem no resultado. É sempre a ÚLTIMA cláusula da instrução SELECT. Por padrão, a ordenação é crescente (ASC); use DESC para inverter.

Analogia: É como pedir ao bibliotecário para entregar os livros em ordem alfabética, do mais antigo para o mais novo, ou do mais caro para o mais barato.

```
SELECT last_name, salary
FROM   employees
ORDER BY salary;
```

Dica: Sem ORDER BY, o SGBD pode retornar as linhas em qualquer ordem — e essa ordem pode mudar entre execuções. Nunca confie na 'ordem natural' do banco.

ASC (Ascending / Crescente)

Ordem crescente: do menor para o maior nos números, do mais antigo para o mais recente nas datas, e em ordem alfabética (A → Z) nos textos. É o padrão do ORDER BY — pode ser omitido.

Analogia: É a ordem do dicionário ou da fila do caixa: começa pelo primeiro e termina pelo último.

```
ORDER BY salary ASC;
ORDER BY salary;      -- mesma coisa
```

DESC (Descending / Decrescente)

Ordem decrescente: do maior para o menor nos números, do mais recente para o mais antigo nas datas, e em ordem alfabética inversa (Z → A) nos textos. Precisa ser escrita explicitamente — não é o padrão.

Analogia: É a contagem regressiva do foguete: 10, 9, 8, 7... ou o ranking dos mais caros, do top do pódio para baixo.

```
ORDER BY salary DESC;
ORDER BY hire_date DESC; -- mais recente primeiro
```

Classificar por Apelido (alias) (ORDER BY usando alias da coluna)

Quando uma coluna calculada recebe um apelido com AS, esse apelido pode ser usado dentro do ORDER BY. Isso torna o código mais legível, especialmente em expressões longas.

Analogia: É como dar um apelido para o colega de trabalho — depois que todos passam a chamar pelo apelido, fica mais fácil referenciar a pessoa.

```
SELECT employee_id, salary * 12 AS anual
FROM   employees
ORDER BY anual;
```

Classificar por Várias Colunas (ORDER BY com lista de colunas)

Permite usar uma coluna como critério principal e outra como critério de desempate. As colunas são listadas separadas por vírgula, e a primeira tem prioridade sobre a segunda, que tem prioridade sobre a terceira, e assim por diante.

Analogia: É como organizar uma turma: primeiro por sala (A, B, C...), e dentro de cada sala, em ordem alfabética dos alunos.

```
-- Por departamento crescente, e dentro de cada
-- departamento, por salario decrescente:
SELECT department_id, last_name, salary
FROM   employees
ORDER BY department_id ASC,
        salary DESC;
```

Dica: ASC e DESC podem ser usados em cada coluna independentemente — uma pode ser ASC enquanto a outra é DESC.

7. Conceitos Transversais

Seleção (Selection) (Filtragem de linhas)

Operação relacional que escolhe quais LINHAS aparecem no resultado, com base em critérios. Em SQL, é realizada pela cláusula WHERE. Não confundir com a projeção, que escolhe colunas.

Analogia: É o filtro de uma planilha do Excel: 'mostrar apenas as vendas acima de R\$ 1.000'. As linhas que não cumprem o critério ficam ocultas.

Dica: Lembre-se: SELECT projeta colunas, WHERE seleciona linhas. Apesar do nome 'SELECT', a seleção de LINHAS é função do WHERE.

Ordem das Cláusulas (SELECT → FROM → WHERE → ORDER BY)

Toda consulta SQL segue uma ordem fixa de cláusulas. Trocar a ordem causa erro de sintaxe. Não importa como o SGBD executa internamente — o que você ESCRIVE deve seguir essa sequência.

Analogia: É como a sequência da receita: primeiro lê o que vai fazer (SELECT), depois pega os ingredientes (FROM), filtra os bons (WHERE) e por fim arruma na travessa (ORDER BY).

SELECT last_name, salary	-- 1. quais colunas?
FROM employees	-- 2. de qual tabela?
WHERE department_id = 50	-- 3. quais linhas?
ORDER BY salary DESC;	-- 4. em que ordem?